

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №6

Выполнил:

Алексеев Андрей

Группа: К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках домашнего задания требовалось настроить автоматическое развёртывание backend-приложения на удалённый сервер. Автодеплой должен запускаться при обновлении кода в репозитории на выбранной ветке и выполнять доставку актуальной версии приложения на сервер без ручного копирования файлов.

В качестве CI/CD-системы были выбраны GitHub Actions. Для деплоя использовалось backend-приложение lab23, ранее развёрнутое на удалённом сервере. Основная идея решения состоит в том, что после push в ветку main workflow подключается к серверу по SSH, обновляет локальную копию репозитория, запускает сценарий деплоя и повторно применяет конфигурацию nginx.

Исходные данные

Для выполнения задания использовался репозиторий ITMO-ACS-Backend-2026-A, в котором находится микросервисный backend-стек lab23. Приложение разворачивается на удалённом сервере в каталоге ~/apps/itmo-acs-backend. Внешняя точка входа в приложение организована через nginx, который проксирует HTTP-запросы на api-gateway.

В рамках работы был подготовлен workflow .github/workflows/lab23-deploy.yml. Также использовались инфраструктурные файлы scripts/lab23/deploy_remote.sh, scripts/lab23/configure_nginx.sh и infra/nginx/lab23.conf. Эти файлы позволяют отделить описание CI/CD-пайплайна от конкретных команд серверного деплоя и настройки reverse проху.

Для безопасного подключения к серверу в репозитории были созданы секреты GitHub Actions. В workflow не хранятся реальные значения адреса

сервера, имени пользователя и приватного SSH-ключа. Вместо этого используются переменные `LAB23_SSH_HOST`, `LAB23_SSH_USER` и `LAB23_SSH_KEY`, которые подставляются в момент выполнения `job`.

Настройка GitHub Actions

Workflow был настроен на событие `push` в ветку `main`. Это означает, что автодеплой запускается только после появления новых изменений в основной ветке репозитория. Такой вариант подходит для учебного backend-проекта, потому что ветка `main` используется как источник актуальной версии приложения для серверного окружения.

Дополнительно в workflow задан фильтр `paths`. Он ограничивает запуск деплоя только теми изменениями, которые относятся к backend-приложению `lab23`, скриптам деплоя, конфигурации `nginx` или самому workflow. За счёт этого пайплайн не запускается на изменения, не влияющие на развёртывание приложения.

В workflow используются следующие основные условия запуска:

- событие — `push`;
- ветка — `main`;
- отслеживаемые пути — `.github/workflows/lab23-deploy.yml`, `scripts/lab23/**`, `infra/nginx/**` и каталог `lab23`.

Job `deploy` выполняется на runner `ubuntu-latest`. При этом сама сборка и запуск приложения происходят не на runner, а на удалённой виртуальной машине. Для подключения к серверу используется action `appleboy/ssh-action@v1.2.2`. Такой подход удобен для текущей схемы, потому что сервер уже содержит окружение для запуска Docker Compose, а GitHub Actions отвечает только за запуск удалённых команд.

Логика автодеплойа

Внутри SSH-сессии workflow задаёт несколько переменных окружения: APP_ROOT, BRANCH и REPO_URL. APP_ROOT указывает на каталог приложения на сервере, BRANCH задаёт ветку main, а REPO_URL содержит адрес GitHub-репозитория. Далее workflow проверяет, существует ли в APP_ROOT локальная копия репозитория.

Если репозиторий на сервере ещё не был клонирован, каталог приложения очищается и создаётся заново, после чего выполняется git clone нужной ветки. Если репозиторий уже существует, workflow выполняет git fetch, переключается на ветку main и обновляет код командой git pull --ff-only. Это позволяет серверу всегда получать именно то состояние кода, которое находится в основной ветке.

После обновления исходного кода запускается скрипт scripts/lab23/deploy_remote.sh. Его задача — выполнить серверный деплой backend-приложения: собрать актуальные Docker-образы, поднять контейнеры через docker compose и удалить устаревшие orphan-контейнеры. Такой сценарий делает деплой повторяемым и не требует вручную вводить длинные команды на сервере.

Затем workflow запускает scripts/lab23/configure_nginx.sh и передаёт ему путь до конфигурации infra/nginx/lab23.conf. Это нужно, чтобы после обновления приложения на сервере была применена актуальная конфигурация nginx. В результате внешний HTTP-доступ продолжает идти через nginx reverse проху, а не напрямую к контейнерам микросервисов.

Для job был задан увеличенный command_timeout. Это важно, потому что первая полная пересборка lab23 на удалённой машине занимает заметное время: внутри Docker собираются несколько backend-сервисов. Увеличение

таймаута предотвращает преждевременное завершение job в ситуации, когда приложение ещё корректно собирается.

Секреты и безопасность

Для подключения к серверу workflow использует секреты репозитория. LAB23_SSH_HOST содержит адрес удалённого сервера, LAB23_SSH_USER — имя пользователя для SSH-подключения, LAB23_SSH_KEY — приватный ключ. Значения этих параметров не записываются в YAML-файл и не попадают в историю коммитов.

Такое разделение важно для безопасности CI/CD. В репозитории хранится только логика деплоя, а чувствительные данные остаются в настройках GitHub Actions. При необходимости ключ или сервер можно заменить в настройках secrets без изменения самого workflow.

Также автодеплой не открывает прямой внешний доступ ко всем внутренним компонентам приложения. Снаружи используется единая точка входа через nginx и api-gateway. Внутренние сервисы, PostgreSQL-контейнеры, Kafka и Elasticsearch остаются частью серверного Docker Compose-контура.

Проверка результата

После настройки workflow ожидаемый сценарий работы выглядит следующим образом: разработчик отправляет изменения в ветку main, GitHub Actions автоматически запускает job deploy, runner подключается к серверу по SSH, обновляет репозиторий в каталоге ~/apps/itmo-acs-backend и запускает серверные скрипты деплоя.

В результате на сервере поднимается актуальная версия backend-приложения lab23. Старые контейнеры заменяются новыми версиями, nginx

остаётся настроенным как reverse proxy, а api-gateway продолжает быть внешней точкой входа в приложение.

Работоспособность результата проверяется по нескольким признакам. Во-первых, workflow должен завершаться без ошибки. Во-вторых, на сервере должны быть подняты контейнеры lab23. В-третьих, внешний адрес приложения должен отвечать через nginx. Дополнительно для проверки API использовался серверный Postman environment и прогон коллекции через Newman.

Контрольный прогон коллекции lab23-microservices.postman_collection.json через Newman подтвердил корректность серверного окружения: было выполнено 65 requests и 65 assertions, количество failed равно 0. Это показывает, что после деплоя backend-приложение доступно и основные API-сценарии работают корректно.

Вывод

В результате выполнения домашнего задания был настроен автодеплой backend-приложения lab23 на удалённый сервер с использованием GitHub Actions. Workflow запускается при push в ветку main и реагирует на изменения в файлах, связанных с приложением, инфраструктурой и деплоем.

Пайплайн подключается к серверу по SSH через защищённые GitHub Secrets, обновляет локальную копию репозитория до актуального состояния ветки main, запускает скрипт серверного деплоя и применяет конфигурацию nginx. Благодаря этому процесс развёртывания становится воспроизводимым и не требует ручного копирования файлов или ручного запуска команд после каждого изменения кода.

Настроенная схема покрывает требования задания: автодеплой привязан к обновлению кода в репозитории, работает для выбранной ветки

main, использует CI/CD-систему GitHub Actions и разворачивает backend-приложение на удалённом сервере. Работоспособность приложения после деплоя подтверждена проверкой внешнего доступа и успешным прогоном API-тестов через Newman.